

BOX : Homework 02

Question 1.

To represent a DBG, we technically only need the list of all nodes, as we can deduce where the edges are from just that. However, if we want our representation to be usable, we will also need a way to store the edges. Since we know that nodes in a DBG have few edges (4 incoming and 4 outgoing at most), it seems efficient to store the edges as an adjacency list.

In my case, I chose to represent DBGs as a dictionary where the keys are the label of the nodes and the values are a pair of lists of labels indicating the edges : given a record label : [list1, list2], any label l in list1 shows an edge from label to l , and any label l in list2 shows an edge from l to label.

Question 2.

To handle the fact that we do not know which strand a read belongs to, I decided that, for each k-mer, the DBG should have a node for both the k-mer and its reverse complement

Question 3.

Let us study the graph we get for the sequence “ACGTACAGT”, for $k = 5$:

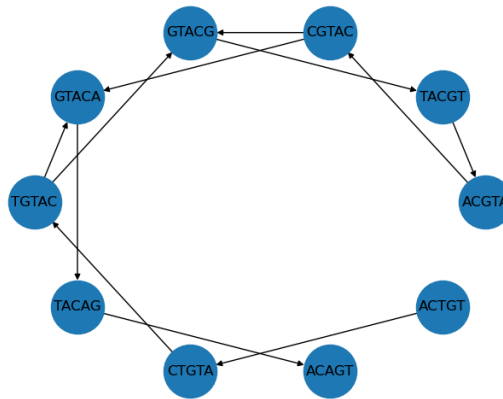


Figure 1: An example graph for the sequence “ACGTACAGT”

We see that this graph is mostly split between two chains : one for the forward strand and one for its reverse complement. We can also see that there happens to be some overlap : some edges still exist between the forward and the backward strand.

Now, let us consider a graph for a slightly different sequence : “ACGCACAGT”, still for $k = 5$:

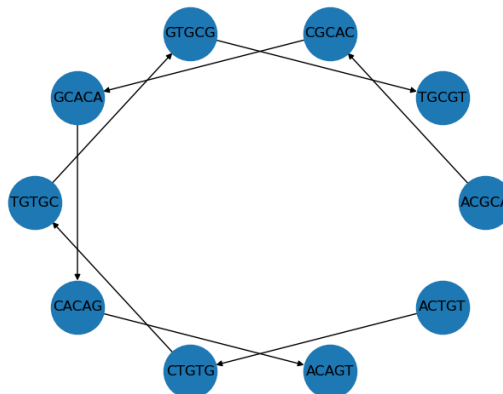


Figure 2: An example graph for the sequence “ACGCACAGT”

This did not change the main structure of the graph (two independent chains), but it did remove all edges between the two strands.

Last, let us look at the graph for a sequence with a repeating 5-mer : “ACGTACGTA”, again for $k = 5$:

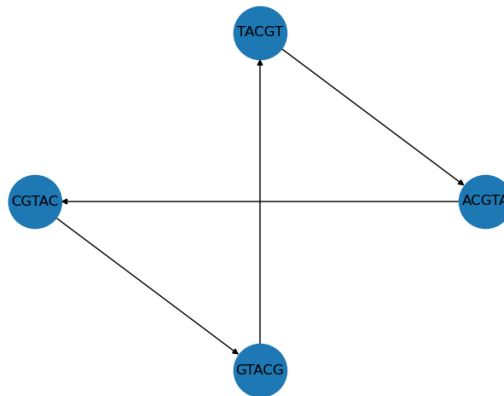


Figure 3: An example graph for the sequence “ACGTACGTA”

This time, we lost all the structure we had for two reasons : first, the sequence is now a loop, so there is no longer a chain, but a cycle. Second, in this sequence, it just so happens that the reverse complement of each 5-mer is already in the original sequence, so both the forward and backward strands are represented by the same nodes.

Question 4.

N/A

Question 5.

In each file, the program processes a total of 2,070,000 k-mers for $k = 31$.

With perfect, forward reads, the program finds 299,730 unique k-mers.

With perfect reads in both direction, the program finds 299,738 unique k-mers.

With reads with 0.1% error, the program finds 424,174 unique k-mers.

With reads with 1% error, the program finds 1,388,536 unique k-mers.

The following graph shows the distribution of their multiplicity :

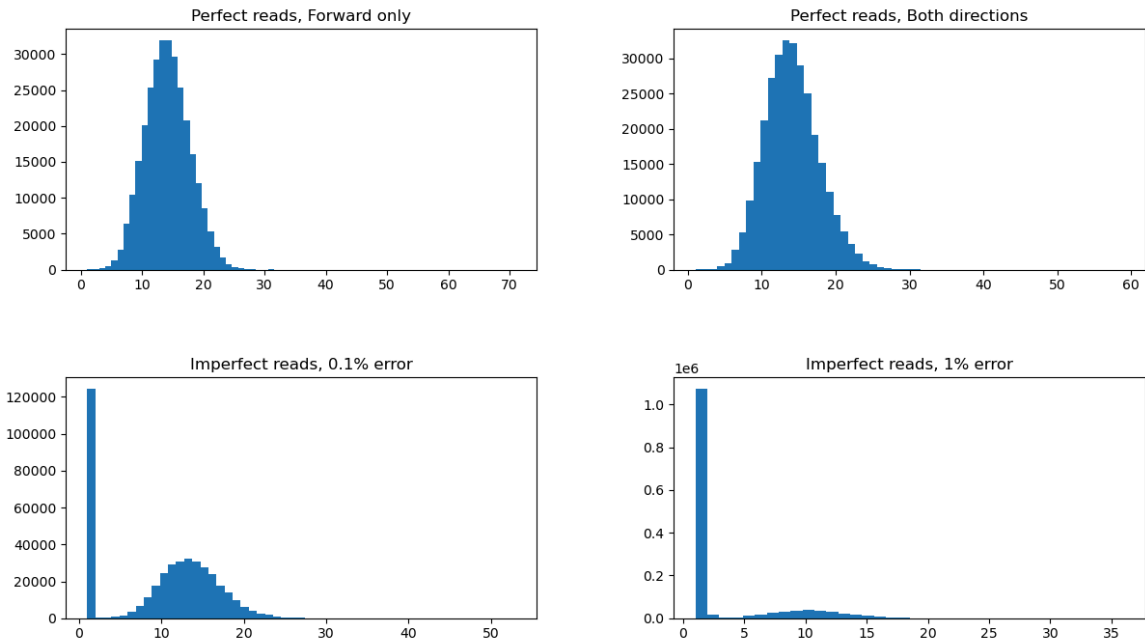


Figure 4: Multiplicity of kmers in each file

Question 6.

N/A

Question 7.

By observing the graphs for the perfect reads, we can see that there would be next to no kmer with multiplicity 1, and very few with multiplicity 2. Therefore, it seems reasonable to use 2 or 3 as a threshold.

Question 8.

N/A

Question 9.

The unitigs for “CGCTCTGTGTGACAAGCCGAAACCGCCCAG” have the following lengths :

	Perfect reads, Forward	Perfect reads, Both strands	Flawed reads, 0.1% error	Flawed reads, 1% error
t=1	5586	5588	46	31
t=2	5575	5581	4811	757
t=3	5574	3242	5580	3095
t=4	5567	3242	5332	2242

There are a few things to note :

- First, it seems that a threshold of 2 or 3 was indeed correct.
- Second, for t=3, we see a significant drop in the size of the unitig for the file with perfect reads and both strands. This is probably because a specific kmer appears a couple less times in the second file than in the first, making it be filtered earlier and cutting the unitig short.

Question 10.

N/A

Question 11.

The sum of the lengths of all sequences can be found in the table below :

	Perfect reads, Forward	Perfect reads, Both strands	Flawed reads, 0.1% error	Flawed reads, 1% error
t=1	300,990	300,998	831,514	4,417,036
t=2	300,964	300,952	301,766	386,724
t=3	300,976	301,110	301,292	304, 944
t=4	301,316	301,468	301,846	311,840